

1) **crear carpeta en la consola**

```
mkdir -p ~/tpo_unidad1  
cd ~/tpo_unidad1
```

2) **creo el archivo sync_service.sh**

```
cat << 'EOF' > sync_service.sh  
#!/bin/bash  
DATA_FILE="/etc/secure_app/transactions.dat"  
  
echo "[INFO] Iniciando el servicio de conciliacion..."  
  
if [ ! -r "$DATA_FILE" ]; then  
    echo "[ERROR FATAL 101]: Fallo critico en el motor transaccional. Contacte a  
    soporte de IT." >&2  
    exit 101  
fi  
  
cat "$DATA_FILE"  
exit 0  
EOF
```

3) **creo la auditoria audit_syscalls.sh**

```
cat << 'EOF' > audit_syscalls.sh  
#!/bin/bash  
chmod +x ./sync_service.sh  
  
echo  
"=====  
echo " 1. INTERCEPTANDO SYSCALLS CON STRACE"  
echo  
"=====  
# -f sigue procesos hijos, -e trace=file intercepta llamadas al sistema de archivos  
strace -f -e trace=file -o auditoria_archivos.log ./sync_service.sh  
  
echo ""  
echo  
"=====  
echo " 2. DIAGNOSTICO: CAUSA RAZ DE DETECTADA A NIVEL KERNEL"  
echo  
"=====  
# Filtramos la llamada sobre el archivo que retornó código de error  
grep "transactions.dat" auditoria_archivos.log  
  
echo ""  
echo  
"=====  
echo " 3. TABLA ESTADISTICA DE LLAMADAS AL SISTEMA (-c)"  
echo  
"=====
```

```
# -c genera el resumen de llamadas, errores y tiempo de procesador en modo
núcleo
strace -c -e trace=execve,openat,read,write,close,newfstatat,faccessat2
./sync_service.sh
EOF
```

4) Otorgo permisos a los dos scripts

```
chmod +x audit_syscalls.sh
chmod +x sync_service.sh
```

5) Por alguna razon si los corro tira este error

strace: exec: Exec format error

```
[student@workstation tpo_unidad1]$ ./audit_syscalls.sh
=====
1. INTERCEPTANDO SYSCALLS CON STRACE
=====
strace: exec: Exec format error

=====
2. DIAGNOSTICO: CAUSA RAIZ DETECTADA A NIVEL KERNEL
=====

=====
3. TABLA ESTADISTICA DE LLAMADAS AL SISTEMA (-c)
=====
strace: exec: Exec format error
% time    seconds  usecs/call   calls    errors syscall
-----
 0.00     0.000000         0         1         1 execve
-----
100.00    0.000000         0         1         1 total
[student@workstation tpo_unidad1]$
```

Para evitar este error tengo que cambiar la cabecera del archivo.

Para eso ejecuto

```
nano ./sync_service.sh
```

elimino la parte de arriba que dice

```
cat << 'EOF' > sync_service.sh
```

y al final que tambien tiene un EOF

Hago lo mismo con ./audit_syscalls.sh}

6) Ejecuto ./audit_syscalls.sh y debería devolver algo así

```
[student@workstation tpo_unidad1]$ ./audit_syscalls.sh
=====
1. INTERCEPTANDO SYSCALLS CON STRACE
=====
[INFO] Iniciando el servicio de conciliacion...
[ERROR FATAL 101]: Fallo critico en el motor transaccional. Contacte a soporte de IT.

=====
2. DIAGNOSTICO: CAUSA RAZA DETECTADA A NIVEL KERNEL
=====
39958 faccessat2(AT_FDCWD, "/etc/secure_app/transactions.dat", R_OK, AT_EACCESS) = -1 ENOENT (No such file or directory)

=====
3. TABLA ESTADISTICA DE SYSCALLS PRINCIPALES (UNIDAD 1)
=====
[INFO] Iniciando el servicio de conciliacion...
[ERROR FATAL 101]: Fallo critico en el motor transaccional. Contacte a soporte de IT.
% time      seconds  usecs/call   calls   errors syscall
-----
54.29      0.000196      5         33      12 openat
22.16      0.000080      2         28      28 newfstatat
17.45      0.000063      2         22      0 close
2.77       0.000010      1          6      0 read
2.77       0.000010      5          2      0 write
0.55       0.000002      2          1      1 faccessat2
0.00       0.000000      0          1      0 execve
-----
100.00     0.000361      3         93      13 total
[student@workstation tpo_unidad1]$
```

Que significa cada cosa?

1) trae el error que seteamos en sync_service.sh (el error generico), lo que queremos hacer con este ejercicio es rastrear el error y a que se debe

2) El diagnostico trae este error:

39909 faccessat2(AT_FDCWD, "/etc/secure_app/transactions.dat", R_OK, AT_EACCESS) = -1 ENOENT (No such file or directory)

Que significa?

- **39909**: Es el **PID** (*Process ID* / Identificador del Proceso) del subprocesso creado para evaluar la condición de lectura
- **faccessat2(...)**: Es la llamada al sistema moderna de Linux que implementa la verificación de permisos y existencia de archivos (la evolución de **access()** para arquitecturas de 64 bits). Corresponde a la línea de Bash **if [! -r "\$DATA_FILE"]**.
- **AT_FDCWD**: Significa *At File Descriptor Current Working Directory* (usar el directorio de trabajo actual para rutas relativas, o procesar la ruta absoluta).
- **"/etc/secure_app/transactions.dat"**: La ruta exacta y recurso físico que el proceso intentó comprobar.
- **R_OK**: Bandera de consulta que verifica si el usuario actual posee permisos de **Lectura (Read OK)**.
- **= -1**: Código de retorno de la syscall. Cuando una llamada al sistema fracasa en el kernel, devuelve **-1** a la biblioteca de usuario (**glibc**).
- **ENOENT (No such file or directory)**: La variable global **errno** fue configurada con la constante **ENOENT** (*Error No Entry*). El Subsistema Virtual de Archivos (**VFS**) del kernel recorrió la tabla de inodos del disco y comprobó que **el archivo o la carpeta /etc/secure_app/ no existe físicamente en el almacenamiento**.

Se demostró de manera empírica que el mensaje corporativo ambiguo (`[ERROR FATAL 101]: Fallo critico en el motor transaccional`) no se debía a una falla del motor ni a una corrupción de memoria, sino a un error físico en el kernel: la syscall `faccessat2` devolvió `-1 ENOENT` por falta del archivo.

el mensaje generico de **ERROR FATAL 101: Fallo critico en el motor transaccional** se debe a que la syscall `faccessat2` devuelve `-1 ENOENT` por falta del archivo `transactions.dat`

Las llamadas al sistema que trae estan seteadas en el archivo `audit_syscalls.sh`

A. `openat` / `open` (y consultas `faccessat2` / `newfstatat`)

- ¿Por qué la ejecuta?

Las aplicaciones en espacio de usuario (Ring 3) no tienen acceso directo a la estructura física de disco ni a las tablas de inodos. Para manipular un archivo, deben solicitar al kernel un descriptor de archivo (`fd`) mediante `open` u `openat`.

En el script, cuando el intérprete ejecuta la prueba de acceso `[ ! -r "$DATA_FILE" ]`, invoca `faccessat2` y `newfstatat` para inspeccionar los metadatos y permisos del archivo antes de intentar abrirlo.

- ¿Por qué aparecen errores en la tabla (`errors = 12`)?

Al iniciarse el proceso, el cargador dinámico busca archivos de configuración y librerías en múltiples rutas estándar (`/lib64/`, `/usr/lib64/`). Aquellas rutas donde el archivo no se encuentra devuelven `-1 ENOENT` y pasan al siguiente directorio. Finalmente, al buscar `/etc/secure_app/transactions.dat`, el kernel comprueba la ausencia del archivo y devuelve `-1`.

B. `read`

- ¿Por qué la ejecuta?

Transfiere secuencias de bytes desde un descriptor de archivo abierto hacia un búfer en la memoria virtual del proceso en espacio de usuario.

- ¿Por qué la ejecuta en este script (`calls = 6`)?

El proceso `/bin/bash` no almacena todo el script completo en memoria en un solo paso; utiliza llamadas `read` sucesivas para ir leyendo del disco las líneas de código que debe parsear e interpretar, además de leer configuraciones de localización y mapeo de bibliotecas compartidas (`libc.so`).

C. `write`

- ¿Por qué la ejecuta?

Es la llamada al sistema encargada de volcar datos desde la memoria del proceso hacia un canal de salida o dispositivo periférico (como la pantalla).

- ¿Por qué se ejecuta en este caso puntual?

Se invoca mediante la función wrapper de `libc` para enviar texto a dos descriptores de archivo distintos:

1. Para imprimir el mensaje informativo en la Salida Estándar (`stdout`, descriptor `1`).

2. Para imprimir **[ERROR FATAL 101]** en la Salida de Error Estándar (`stderr`, descriptor `2`) a través de la redirección `>&2`.

D. `close`

- **¿Por qué la ejecuta?**
Cierra los descriptores de archivos que el proceso ya no necesita utilizar.
- **Rol en el sistema operativo:**
La tabla de descriptores de un proceso es finita. Cuando el intérprete termina de leer las librerías dinámicas o archivos de comandos, ejecuta `close(fd)` para liberar la entrada en su tabla local y permitir que el kernel decremente el contador de referencias sobre la estructura de archivo abierta.

E. `execve`

- **¿Por qué la ejecuta?**
Es la llamada primordial de administración de procesos en UNIX/Linux.
- **Rol en el sistema operativo:**
Cuando el shell padre ejecuta el script, crea un proceso nuevo y este invoca `execve` pasándole como argumento `/bin/bash` y `./sync_service.sh`. El kernel comprueba el encabezado *Shebang* (`#!/bin/bash`), sustituye el espacio de direcciones anterior por la nueva imagen de memoria del intérprete y sitúa el Contador de Programa (PC) en la primera instrucción a ejecutar

El archivo `transaction.dat` (`DATA_FILE="/etc/secure_app/transactions.dat"`) ubicado en la línea 3 de `sync_service.sh` es la trampa del ejercicio, al nunca ser creado, el error generico en lugar de traer un 'Error fatal', debería avisar que el archivo que intenta acceder, no existe.

EXTRA: Otra opción en lugar de crear los archivos, es traerlo desde github.

El repositorio es: [Lucasalexisc/sistemas-operativos](https://github.com/Lucasalexisc/sistemas-operativos)

- 1) traer el repositorio: `git clone https://github.com/Lucasalexisc/sistemas-operativos`
- 2) `cd sistemas-operativos`
`ls -la`
- 3) Si hago cambios en la pc local, traigo los archivos con
`git pull origin main`
- 4) Para ver las carpetas
`ls -l`